# PLLM: Pseudo-Labeling Large Language Models for CAD Program Synthesis

Anonymous CVPR submission

Paper ID *****

## Abstract

*Recovering Computer-Aided Design (CAD) programs from 3D geometries is a widely studied problem. Recent advances in large language models (LLMs) have enabled progress in CAD program synthesis, but existing methods rely on supervised training with paired shape–program data, which is often unavailable.*

*We introduce PLLM, a self-training framework for CAD program synthesis from unlabeled 3D shapes. Given a pre-trained CAD-capable LLM and a shape dataset, PLLM iteratively samples candidate programs, selects high-fidelity executions, and augments programs to construct synthetic program–shape pairs for fine-tuning. We experiment on adapting CAD-Recode from DeepCAD to the unlabeled ABC dataset show consistent improvements in geometric fidelity and program diversity.*

## 1. Introduction

Computer-Aided Design (CAD) is the industry standard for 3D modeling in engineering and manufacturing. Designers typically construct models through a sequence of parametric operations, which, when executed, produce boundary representations (B-reps) of 3D geometry. The inverse problem of recovering a CAD program from a given shape is also extensively studied. Recovering the program enables semantic editing, programmatic modification, and compact representation of 3D models.

Previous approaches address this inverse problem by training lightweight neural networks to predict CAD operations and their corresponding parameters [1, 5, 11, 52, 55]. More recently, large language models (LLMs) have been explored for this reverse-engineering task due to their strong symbolic reasoning abilities and rapid progress in program synthesis [27, 30, 34, 43, 44]. However, existing methods depend on supervised training with ground-truth CAD programs. Such annotations are expensive, dataset-specific, and often unavailable, creating a major bottleneck for scaling CAD program synthesis to new domains and new CAD languages.

In this work, we introduce a new framework to address this the lack of data. Rather than relying on labeled programs, we derive supervision from the data generation process itself. Formally, we assume a pre-trained LLM $p(z \mid x, \mathcal{L})$ that generates a CAD program $z$ in language $\mathcal{L}$ from a shape $x \sim \mathcal{S}$. Given a new shape distribution $\mathcal{S}^*$ without program annotations, our goal is to adapt the model by automatically generating informative training data from $\mathcal{S}^*$ itself.

Our observation is that the pre-trained LLM already encodes substantial knowledge about CAD structure and can produce plausible programs even for unseen domains, though they might be suboptimal. We therefore treat model outputs not as final predictions but as starting point for data synthesis. Our framework samples programs, executes them, compares the resulting geometry to the input shape, and selectively retains high-fidelity results. We further expand this data by programmatic edits and variations, producing enriched program–shape pairs that provide stronger supervision than the original predictions alone. Through iterative self-training, the model gradually improves using this growing synthetic dataset.

Specifically, we instantiate this framework using CAD-Recode [34], trained on DeepCAD [49] to generate programs in the CadQuery language. We then adapt it to the ABC dataset [25], a widely used shape collection that lacks ground-truth CAD programs. This setting highlights the ability of our approach to synthesize useful supervision where none exists.

In summary, we propose a novel method to fine-tune existing LLMs for improved CAD program synthesis for new domain in the absence of ground-truth supervision. Our contributions are as follows:

- We introduce PLLM, a self-training framework that synthesizes and enriches CAD programs to construct supervision for unlabeled 3D datasets.
- We propose a data synthesis strategy that expands model outputs through sampling and programmatic edits, generating diverse and informative program–shape pairs.
- We demonstrate that synthetic self-training improves geometric fidelity when adapting CAD-Recode from Deep-

CAD to the unlabeled ABC dataset.

## 2. Related Works

### 2.1. Self Supervised Training

Our work lies in the broader area of unsupervised and weakly-supervised learning [7, 46], where the central challenge is how to obtain useful supervision when labeled data is scarce or unavailable. A common solution in such settings is reinforcement learning (RL) [32, 38, 39, 46, 57], which treats supervision as a reward signal. However, CAD programs involve discrete, non-differentiable operations and expensive execution, making RL-based optimization unstable and inefficient for our setting. Instead, we adopt a self-training paradigm, where the model generates its own supervision. Self-training has a long history in weakly-supervised learning [31, 35, 53], and recent work shows that combining self-training with data augmentation and synthetic data generation can substantially improve neural models across domains [18, 22, 58]. Our approach builds on this perspective by treating model outputs as a source of synthetic data that can be filtered, expanded, and reused for training.

In visual program synthesis, self-training has recently emerged as an effective strategy for learning without ground-truth programs [15, 19, 20]. Our method is closely related to execution-guided learning [8, 12–14], where program execution provides feedback on correctness. In our case, execution serves as a mechanism to evaluate and curate synthetic programs, allowing us to retain high-quality program–shape pairs as supervision.

PLAD [21] proposes a bootstrapped learning framework in which a pre-trained generator produces candidate programs that are then used to fine-tune the model. Our approach follows a similar high-level philosophy but extends it toward richer data synthesis: rather than only selecting successful generations, we further expand and refine programs through programmatic edits and variations. This transforms initial predictions into a growing and increasingly informative synthetic dataset. We instantiate this idea in the context of CAD program synthesis using a pre-trained LLM as the generator.

### 2.2. Learning to Recover CAD Programs

Our work also relates to the broader goal of reverse CAD engineering from diverse input modalities, including voxel grids [26, 36, 40], point clouds [10, 17, 28, 37, 41, 48, 49], and boundary representations [52]. Early approaches relied on heuristics or lightweight neural networks, while more recent works explore large language models due to their symbolic reasoning and program synthesis capabilities [2–4, 16, 29, 33, 45, 50, 54, 56]. Our method belongs to this family of reverse CAD approaches.

However,a common limitation for existing methods [23, 24, 27, 34, 43, 51] is that they rely on datasets containing paired ground-truth CAD programs and shapes. Such paired data is expensive to obtain and scarce in practice. Many large-scale CAD repositories [6, 25, 42, 47] provide high-quality geometry but lack program annotations. This data imbalance limits the scalability of supervised reverse CAD systems.

Our approach addresses this challenge from a data synthesis perspective. Rather than requiring paired supervision, we treat unlabeled CAD collections as raw material for generating synthetic program–shape pairs. By leveraging a pre-trained model (CAD-Recode [34]) to propose programs and then expanding, filtering, and refining these programs through execution feedback and programmatic edits, we construct a growing synthetic dataset that enables adaptation to new domains without manual annotations.

## 3. Method

In this section, we formally describe the PLLM framework, which treats CAD program synthesis as a process of iterative data construction and model refinement. The framework takes the following components as input:

- **(1) Pre-trained LLM:** A model $p(z \mid x, \mathcal{L})$ capable of generating CAD programs $z$ from input shapes $x$ using the language $\mathcal{L}$, where $x$ is drawn from a source distribution $\mathcal{S}$.
- **(2) Target shape dataset:** A dataset of shapes $\mathcal{S}^*$ from a target distribution that differs from $\mathcal{S}$ and lacks program annotations.
- **(3) Black-box executor:** An executor $\mathcal{E}$ that executes a program $z$ to produce its corresponding 3D geometry.

Rather than assuming access to ground-truth programs, PLLM constructs supervision automatically from $\mathcal{S}^*$. The pre-trained model generates candidate programs for shapes in $\mathcal{S}^*$, which are executed and evaluated against the input shapes. High-quality executions are treated as synthetic supervision and accumulated into a training set that supports iterative refinement of the model.

The objective of PLLM is to adapt the pre-trained model to the new distribution $\mathcal{S}^*$ by leveraging this synthetic supervision. Through iterative self-training, we obtain an updated model $p'$. For an input shape $x^* \in \mathcal{S}^*$, a sampled program $z^* \sim p'(z \mid x^*, \mathcal{L})$ should execute to a geometry $\mathcal{E}(z^*)$ that better matches the input shape. We quantify this improvement using a geometric reward metric (Chamfer Distance), where lower distance indicates higher fidelity.

We illustrate the overall PLLM procedure in Figure 1. PLLM adapts $p(z \mid x, \mathcal{L})$ to the target distribution $\mathcal{S}^*$ through an iterative data synthesis and self-training pipeline consisting of four key steps. First, the pre-trained model is used to sample multiple candidate programs for each input

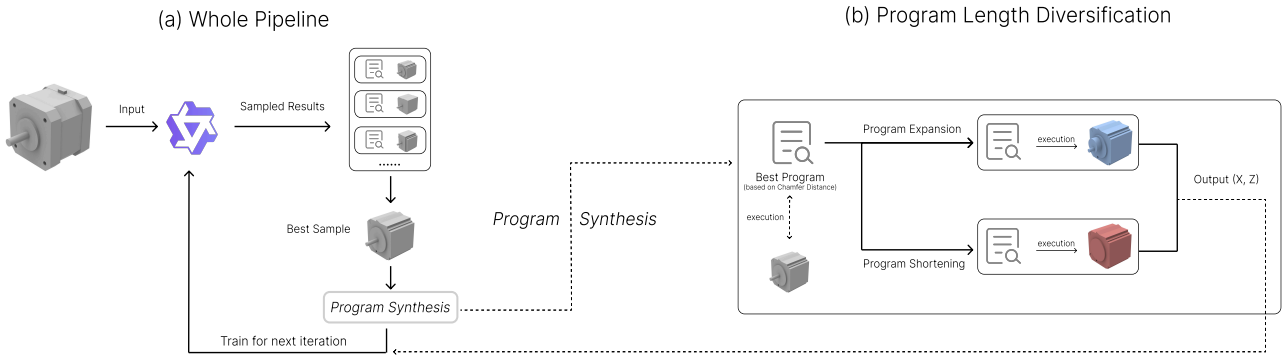



Figure 1. We show the overall pipeline in (a). At each iteration, the model first takes an input shape and samples multiple candidate programs. The selection algorithm then identifies the best program–shape pairs, which are used for training in the next iteration. (b) illustrates the details of the program length diversification process, where we perform both program expansion and shortening to create additional variants. The edited programs serve as labels $Z$, and their corresponding executions are treated as inputs $X$ to form the new training dataset.

shape $x^* \in \mathcal{S}^*$ (Section 3.1), producing diverse program hypotheses. Second, these candidates are executed and evaluated against the input shape, and the highest-fidelity programs are retained as initial synthetic supervision based on Chamfer Distance (Section 3.1). Third, programmatic edits are applied to the selected programs to synthesize additional valid variants, enriching the program–shape pairs and exposing the model to a broader space of feasible solutions (Section 3.2). Finally, the LLM is fine-tuned on this expanded synthetic dataset (Section 3.3).

Across iterations, these steps form a self-reinforcing data generation cycle: as the model improves, it produces higher-quality programs, which in turn yield more accurate and diverse synthetic training pairs. These improved pairs provide stronger supervision for subsequent updates, progressively aligning the model with the target distribution $\mathcal{S}^*$.

### 3.1. Program Sampling

Given an input shape $x^*$, the pre-trained LLM $p(z \mid x^*, \mathcal{L})$ generates $k = 10$ candidate programs $\{z_i\}_{i=1}^{k}$ via stochastic decoding. We use nucleus sampling (top-$p = 0.8$, top-$k = 30$) with temperature 1.2 to encourage moderate diversity.

Each candidate is executed and compared to the input shape using Chamfer Distance. The program with the lowest distance is selected as the representative program $z^*$. If multiple candidates yield nearly identical reconstructions (difference $< 10^{-4}$), we prefer shorter programs to promote concise representations.

### 3.2. Program-Level Data Augmentation

The pre-trained model may not fully capture the range of program structures required by the new distribution $\mathcal{S}^*$, particularly when shapes exhibit varying procedural complexity (see analysis in Section 5.3). As a result, directly using the model's outputs can produce a narrow distribution of program lengths and structures.

To address this, we perform program-level data augmentation by synthetically expanding or shortening selected programs.

**Program Expansion** We extend programs by appending additional operations to existing workspaces or by spawning new workspaces with procedurally generated sketch–feature sequences. We cap the total number of workspaces at $W_{\max} = 5$ to maintain compact and executable programs while gradually increasing structural complexity.

**Program Shortening** We also generate shorter variants by removing top-level boolean operations (`union`, `cut`, `intersect`) while preserving syntactic validity. This produces more concise programs without drastically altering the resulting geometry.

These transformations create additional valid program–shape pairs that preserve geometric consistency while varying procedural structure. As a result, the synthetic dataset spans a broader range of program lengths and complexities, exposing the model to a wider spectrum of structural patterns and improving generalization to shapes with diverse complexity levels. This process is illustrated in Figure 1(b).

### 3.3. Training Data Pairs

We perform LoRA fine-tuning on the LLM using the synthetically constructed program–shape pairs, where both extended and shortened programs serve as $Z$ and their corre-

sponding executions serve as $X$. A key advantage of this design is that each $(X, Z)$ pair is perfectly consistent: the shape $X$ is the direct execution result of program $Z$. This eliminates label noise and provides reliable supervision during fine-tuning.

By incorporating both extended and shortened programs, the synthetic dataset spans a broader range of program lengths and structural complexities. This enriched supervision improves the model's ability to generalize across varying procedural patterns. In practice, this strategy maintains training stability while progressively expanding the model's capacity to generate programs with diverse lengths and structures through iterative updates. We present additional experiments using alternative data-pair configurations in Section 5.4.

# 4. Implementation

We use CAD-Recode [34], pre-trained on DeepCAD [49], as our base model. The target domain $\mathcal{S}^*$ is the ABC dataset [25]. Program execution is performed using CadQuery and its interpreter [9].

## 4.1. CAD-Recode

We adopt CAD-Recode [34] as our base model. CAD-Recode performs reverse CAD by mapping an input point cloud to executable CadQuery code. It consists of a point-cloud encoder that produces feature embeddings and a language-model decoder that generates CAD programs conditioned on these embeddings.

CAD-Recode is originally trained on the DeepCAD dataset using sketch–extrude programs. Since ABC shapes often require more diverse operations, we approximate them using sketch–extrude sequences rather than exact reconstruction. We also increase the maximum program length from 768 to 1200 tokens and apply our program diversification strategy to encourage longer and more detailed programs.

## 4.2. LoRA Fine-Tuning

We fine-tune CAD-Recode using Low-Rank Adaptation (LoRA) to support longer and more complex program generation. LoRA is applied to the middle transformer layers (layers 4–8) while keeping the remaining layers frozen. We use rank $r = 8$, $\alpha = 32$, and dropout $p = 0.1$, applied to both self-attention and MLP projections.

## 4.3. Computational Cost

We use 75,000 shapes from the first 15 ABC batches (5,000 per batch). Of these, 71,784 yield executable programs and are used for experiments.

Training is performed on four NVIDIA L40S GPUs (48GB each) and an AMD EPYC 7R13 CPU. Six self-training iterations take 150 hours total ( 25 hours/iteration): 12 hours for sampling, 10 hours for execution and selection, and 2 hours for four epochs of LoRA fine-tuning.

# 5. Results and Evaluations

We sample point clouds from shapes in the ABC dataset and process them through the PLLM pipeline. We report qualitative comparisons against CAD-Recode (Figure 4) and show results across training iterations (Figure 5). Quantitative evaluations include Chamfer Distance, Intersection over Union (IoU), and program length (Figure 2; Sections 5.1, 5.2, and 5.3).

## 5.1. Chamfer Distance Across Iterations

We report the best, average, and worst Chamfer Distances across iterations in Figure 2(a). Distances are computed after normalizing both predicted and input shapes to a unit bounding box ($1^3$) and scaling by $10^3$. The best and worst values are the mean Chamfer Distance of the top 10 and bottom 10 shapes per iteration, respectively, while the average is computed over all shapes. Chamfer Distance decreases consistently over the first four iterations, indicating improved geometric fidelity. Afterward, improvements plateau, which we attribute to the limited operation set of the base model CAD-Recode.

## 5.2. IoU Across Iterations

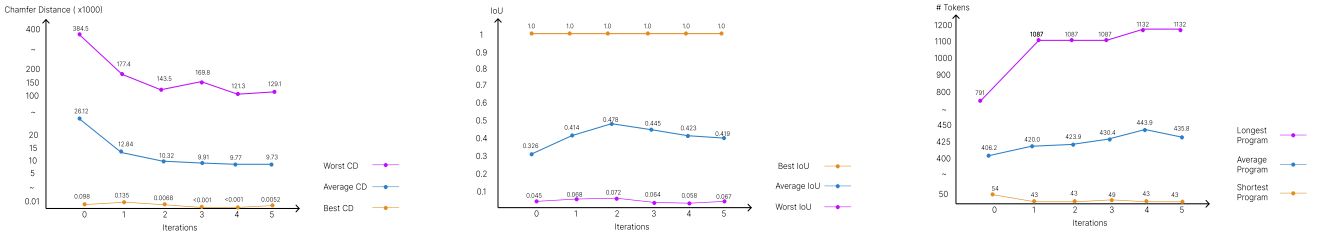
Another interesting metric to consider is the IoU across iterations (Figure 2(b)), which is not directly optimized in our framework. We do not intentionally select programs with high IoU, as our objective focuses on minimizing the Chamfer Distance (CD). While IoU measures volumetric overlap, CD evaluates surface alignment between the generated and target shapes. In our results, we observe that IoU increases during the first two iterations but decreases in later ones. This behavior arises because IoU is not explicitly used as a reward signal—thus, as the model focuses more on lowering CD, it may overfit surface alignment without necessarily improving volumetric consistency.

## 5.3. Program Length Distance Across Iterations

We analyze how average, longest, and shortest program lengths evolve across iterations in Figure 2(c). Initially, average length increases, allowing finer shape generation. The baseline model, CAD-Recode, is limited to 768 tokens. When this cap is raised to 1200 tokens at iteration 0, program length grows slightly. From iteration 2 onward, as longer programs are added through expansion (see Section 3.2), the maximum length rises markedly, improving the model's capacity to represent detailed geometries.

## 5.4. Experiments with Different Pseudo Label Pairs

To fine-tune the model, pseudo program–shape pairs should ideally: (1) use high-quality programs, (2) provide exe-




(a) Chamfer Distance across Iterations (b) IoU across Iterations (c) Program Length across Iterations

Figure 2. We compare quantitative results across iterations: (a) Chamfer Distance, (b) IoU, and (c) Program Length.

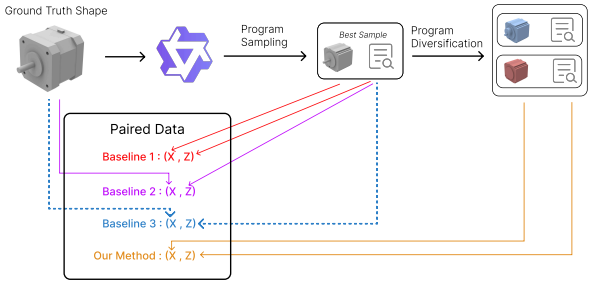



Figure 3. Overview of different baseline strategies compared in our study. The figure illustrates how each baseline constructs its $(X, Z)$ training pairs. Baseline 1 uses the generated program and its execution; Baseline 2 uses the input shape and its best generated program; and Baseline 3 samples within each batch, selecting only the top 20% of high-performing pairs. Our proposed method further introduces program expansion and shortening to generate paired data $(X, Z)$ that better align with the target distribution.

cutable program–shape consistency, (3) reflect the target shape distribution, and (4) introduce informative program variations. In practice, pseudo-labeling methods cannot satisfy all criteria simultaneously, leading to trade-offs.

Our approach selects top-performing programs (Criterion 1) and uses diversified programs with their executions for training, ensuring program–shape consistency and additional structural variation (Criteria 2 and 4). We partially satisfy Criterion 3 by anchoring supervision to executions derived from target-domain shapes.

We compare against alternative pairing strategies (Figure 3). Final-iteration results are summarized in Table 1, where our method achieves the best overall performance.

#### 5.4.1. Baseline 1: (best sample, execution) pair

This baseline trains on generated programs paired with their executions. Performance degrades because training data increasingly drifts from the target shape distribution.

#### 5.4.2. Baseline 2: (best sample, input shape) pair

This baseline pairs generated programs with input shapes. It yields improvements but introduces supervision noise since programs do not exactly reconstruct the paired shapes.

Table 1. Comparison of different pseudo-label and program pairing strategies evaluated at the final iteration. Our proposed method, which uses paired synthetic programs and their executions for training, achieves the lowest Chamfer Distance and demonstrates the most consistent performance improvement across iterations.

| Sampling Method | Final Average CD |
|---|---|
| Our Method | 9.73 |
| CAD-Recode | 26.12 |
| Baseline 1 (best sample, its execution) | 28.24 |
| Baseline 2 (best sample, input shape) | 10.28 |
| Baseline 3 (In Batch Sampling) | 22.84 |

#### 5.4.3. Baseline 3: In Batch Sampling

This variant of Baseline 2 trains only on the top 20% of samples per batch. While top-performing shapes improve, lower-performing cases receive no updates, limiting overall gains.

## 6. Conclusion

We presented PLLM, a data-centric self-training framework for CAD program synthesis that treats unlabeled shapes as a source for constructing synthetic supervision. By iteratively generating, curating, and enriching CAD programs, PLLM converts raw shape collections into a growing program–shape dataset that supports model improvement without requiring paired annotations.

Our pipeline combines program sampling, execution-based curation, and program-level augmentation to expand both the quality and diversity of synthetic programs. Our evaluations demonstrate that this synthetic data generation process consistently improves geometric fidelity and program diversity, allowing PLLM to outperform the baseline CAD-Recode model with lower Chamfer Distances across iterations while maintaining valid and interpretable CAD code.

A major limitation of our approach is computational cost. Iterative data synthesis requires repeated sampling,

execution, and filtering of programs. However, this cost reflects the trade-off of replacing manual annotation with automated supervision construction. As program executors and sampling efficiency improve, we expect such data-centric approaches to become increasingly practical for scaling CAD program learning to large unlabeled repositories.

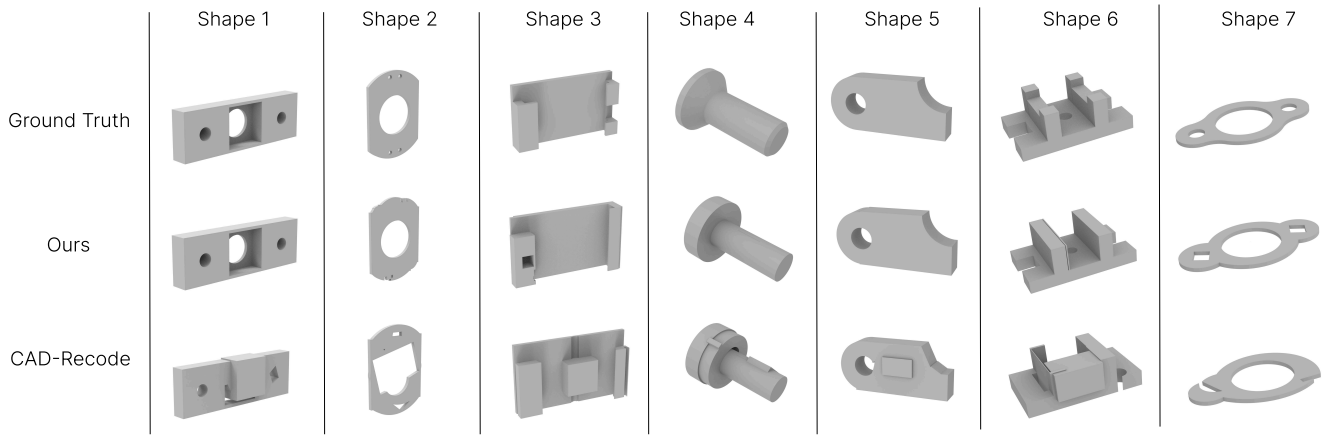



Figure 4. Comparison between our results and those produced by CAD-Recode, which correspond to the outputs from the first iteration of our framework

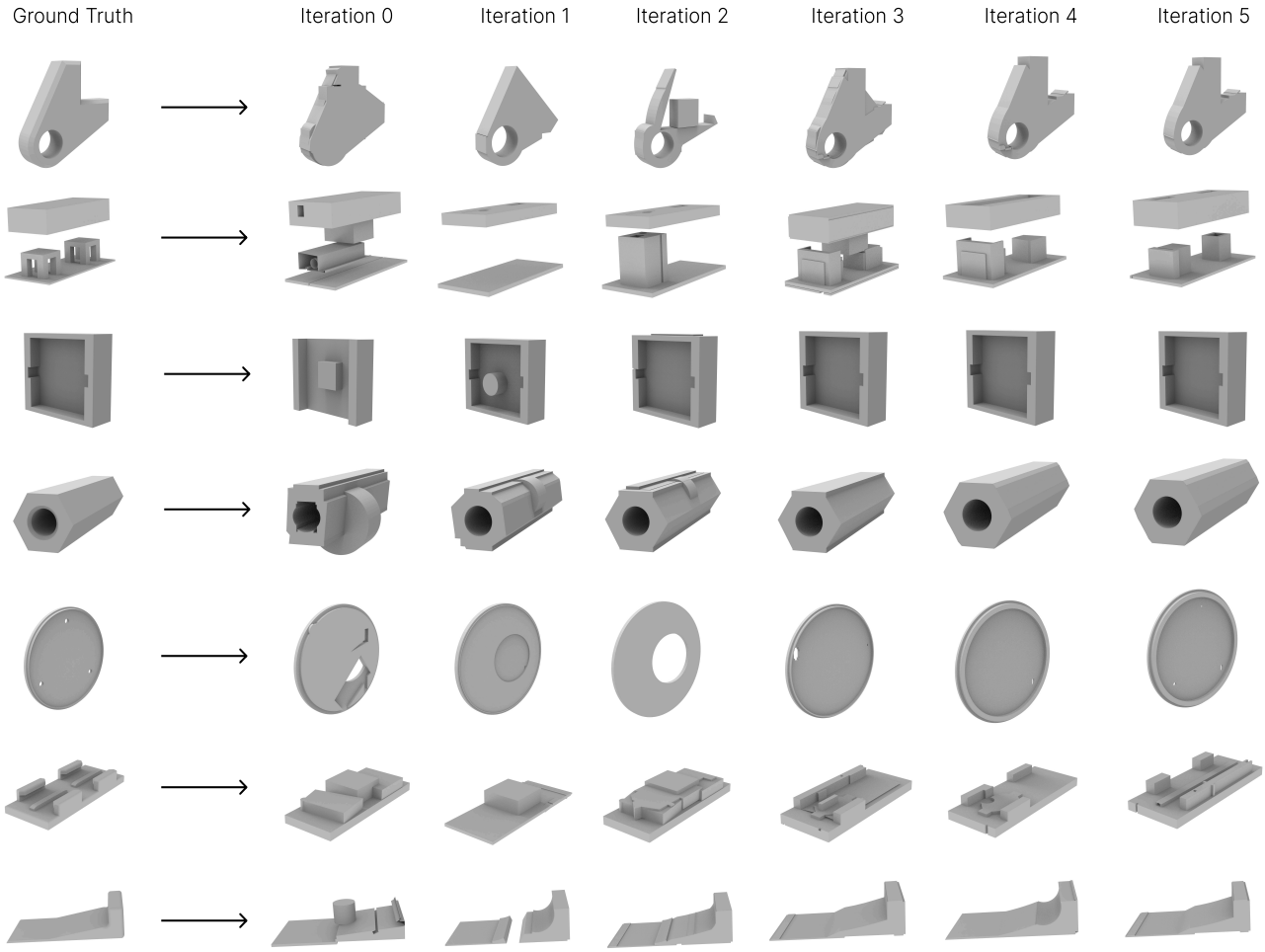



Figure 5. Results across different iterations, showing that the generated shapes gradually improve in quality as training progresses

# References

[1] Sk Aziz Ali, Mohammad Sadil Khan, and Didier Stricker. Brep boundary and junction detection for cad reverse engineering. In *IEEE International Conference on Computing and Machine Intelligence (ICMI)*, 2024. 1

[2] Kamel Alrashedy, Pradyumna Tambwekar, Zulfiqar Zaidi, Megan Langwasser, Wei Xu, and Matthew C. Gombolay. Generating cad code with vision-language models for 3d designs. *arXiv preprint arXiv:2410.05340*, 2024. arXiv:2410.05340. 2

[3] Kamel Alrashedy, Pradyumna Tambwekar, Zulfiqar Zaidi, Megan Langwasser, Wei Xu, and Matthew C. Gombolay. Generating cad code with vision-language models for 3d designs. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025. Preprint available via IEEE Xplore (Document 10890248).

[4] Akshay Badagabettu, Sai Sravan Yarlagadda, and Amir Barati Farimani. Query2cad: Generating cad models using natural language queries. *CoRR*, abs/2406.00144, 2024. 2

[5] Pal Benko and J *et al.* Faigl. Algorithms for reverse engineering boundary representation (b-rep) solid models. Technical report, Berkeley EECS / UC Berkeley, 2001. 1

[6] Pratyush Bharadwaj, Paul Willberg, Faizan Ahmad, Adeel Ahmad, et al. Simjeb: Simulated joint engineering benchmark. https://simjeb.github.io, 2023. Accessed: 2025-07-12. 2

[7] Rudy Bunel, Matthew Hausknecht, Jacob Devlin, Rishabh Singh, and Pushmeet Kohli. Leveraging grammar and reinforcement learning for neural program synthesis. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2018. 2

[8] Xinyun Chen, Chang Liu, and Dawn Song. Execution-guided neural program synthesis. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2019. 2

[9] CadQuery developers. Cadquery — a python parametric cad scripting framework. 4

[10] Tao Du, Jeevana Priya Inala, Yewen Pu, Andrew Spielberg, Adriana Schulz, Daniela Rus, Armando Solar-Lezama, and Wojciech Matusik. Inversecsg: Automatic conversion of 3d models to csg trees. *ACM Transactions on Graphics (Proc. SIGGRAPH Asia*, 37(6), 2018. 2

[11] Elona Dupont, Kseniya Cherenkova, Anis Kacem, Sk Aziz Ali, Ilya Arzhannikov, Gleb Gusev, and Djamila Aouada. Cadops-net: Jointly learning cad operation types and steps from boundary-representations. In *arXiv preprint arXiv:2208.10555*, 2022. 1

[12] Kevin Ellis, Daniel Ritchie, Armando Solar-Lezama, and Josh Tenenbaum. Learning to infer graphics programs from hand-drawn images. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018. 2

[13] Kevin Ellis, Maxwell Nye, Yewen Pu, Felix Sosa, Josh Tenenbaum, and Armando Solar-Lezama. Write, execute, assess: Program synthesis with a repl. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[14] Kevin Ellis, Catherine Wong, Maxwell Nye, Mathias Sable-Meyer, Luc Cary, Lucas Morales, Luke Hewitt, Armando Solar-Lezama, and Joshua B. Tenenbaum. Dreamcoder: Growing generalizable, interpretable knowledge with wake–sleep bayesian program learning. *arXiv preprint arXiv:2006.08381*, 2020. 2

[15] Aditya Ganeshan, R. Kenny Jones, and Daniel Ritchie. Improving unsupervised visual program inference with code rewriting families. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023. 2

[16] Yandong Guan, Xilin Wang, Xingxi Ming, Jing Zhang, Dong Xu, and Qian Yu. Cad-coder: Text-to-cad generation with chain-of-thought and geometric reward. *arXiv preprint arXiv:2505.19713*, 2025. arXiv:2505.19713. 2

[17] Haoxiang Guo, Shilin Liu, Hao Pan, Yang Liu, Xin Tong, and Baining Guo. Complexgen: Cad reconstruction by b-rep chain complex generation. *ACM Transactions on Graphics (SIGGRAPH)*, 41(4), 2022. 2

[18] Junxian He, Jiatao Gu, Jiajun Shen, and Marc'Aurelio Ranzato. Revisiting self-training for neural sequence generation. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2020. 2

[19] Robert Jones, Daniel Ritchie, and Armando Solar-Lezama. Shapecoder: Discovering abstractions for visual programs from unstructured primitives. *ACM Transactions on Graphics (TOG)*, 42(4):1–13, 2023. 2

[20] Robert Jones, Shoubhik Bhat, Daniel Ritchie, and Armando Solar-Lezama. Learning to edit visual programs with self-supervision. *arXiv preprint arXiv:2406.02383*, 2024. 2

[21] R. Kenny Jones, Homer Walke, and Daniel Ritchie. Plad: Learning to infer shape programs with pseudo-labels and approximate distributions. *CVPR*, 2022. Revised version v4, 22 Mar 2022. 2

[22] Jacob Kahn, Ann Lee, and Awni Hannun. Self-training for end-to-end speech recognition. In *ICASSP 2020 - 2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 7084–7088. IEEE, 2020. 2

[23] Mohammad Sadil Khan, Elona Dupont, Sk Aziz Ali, Kseniya Cherenkova, Anis Kacem, and Djamila Aouada. Cad-signet: Cad language inference from point clouds using layer-wise sketch instance guided attention. *CVPR*, 2024. 2

[24] Muhammad Tayyab Khan, Lequn Chen, Ye Han Ng, Wenhe Feng, Nicholas Yew Jin Tan, and Seung Ki Moon. Leveraging vision-language models for manufacturing feature recognition in cad designs. *arXiv preprint arXiv:2411.02810*, 2024. 2

[25] Sebastian Koch, Albert Matveev, Zhongshi Jiang, Francis Williams, Alexey Artemov, Evgeny Burnaev, Marc Alexa, Denis Zorin, and Daniele Panozzo. Abc: A big cad model dataset for geometric deep learning. In *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019. 1, 2, 4

[26] Joseph George Lambourne, Karl Willis, Pradeep Kumar Jayaraman, Longfei Zhang, Aditya Sanghi, and Kamal Rahimi Malekshan. Reconstructing editable prismatic cad from rounded voxel models. In *SIGGRAPH Asia Conference Papers*, 2022. 2

[27] Jiahao Li, Weijian Ma, Xueyang Li, Yunzhong Lou, Guichun Zhou, and Xiangdong Zhou. Cad-llama: Leveraging large language models for computer-aided design parametric 3d model generation. *arXiv preprint arXiv:2505.04481*, 2025. 1, 2

[28] Yujia Liu, Anton Obukhov, Jan Dirk Wegner, and Konrad Schindler. Point2cad: Reverse engineering cad models from 3d point clouds. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1540–1550, 2024. 2

[29] Liane Makatura, Michael Foshey, Bohan Wang, Felix Hähnlein, Pingchuan Ma, Bolei Deng, Megan Tjandrasuwita, Andrew Spielberg, Crystal Elaine Owens, Peter Yichen Chen, Allan Zhao, Amy Zhu, Wil J. Norton, Edward Gu, Joshua Jacob, Yifei Li, Adriana Schulz, and Wojciech Matusik. How can large language models help humans in design and manufacturing? *arXiv preprint arXiv:2307.14377*, 2023. arXiv:2307.14377. 2

[30] Dimitrios Mallis, Ahmet Serdar Karadeniz, Sebastian Cavada, Danila Rukhovich, Niki Foteinopoulou, Kseniya Cherenkova, Anis Kacem, and Djamila Aouada. Cad-assistant: Tool-augmented vllms as generic cad task solvers. *ICCV*, 2025. arXiv:2412.13810. 1

[31] David McClosky, Eugene Charniak, and Mark Johnson. Effective self-training for parsing. In *Proceedings of the Human Language Technology Conference of the NAACL, Main Conference*, pages 152–159, New York City, USA, 2006. Association for Computational Linguistics. 2

[32] Volodymyr Mnih, Adrià Puigdomènech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *International Conference on Machine Learning*, pages 1928–1937, 2016. 2

[33] Felix Ocker, Stefan Menzel, Ahmed Sadik, and Thiago Rios. From idea to cad: A language model-driven multi-agent system for collaborative design. *arXiv preprint arXiv:2503.04417*, 2025. 2

[34] Danila Rukhovich, Elona Dupont, Dimitrios Mallis, Kseniya Cherenkova, Anis Kacem, and Djamila Aouada. Cad-recode: Reverse engineering cad code from point clouds. *ICCV*, 2025. 1, 2, 4

[35] H. Scudder. Probability of error of some adaptive pattern-recognition machines. *IEEE Transactions on Information Theory*, 11(3):363–371, 1965. 2

[36] Gopal Sharma, Rishabh Goyal, Difan Liu, Evangelos Kalogerakis, and Subhransu Maji. Csgnet: Neural shape parser for constructive solid geometry. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018. 2

[37] Gopal Sharma, Difan Liu, Evangelos Kalogerakis, Subhransu Maji, Siddhartha Chaudhuri, and Radomír Měch. Parsenet: A parametric surface fitting network for 3d point clouds. In *Proc. European Conference on Computer Vision (ECCV)*, 2020. 2

[38] David Silver, Guy Lever, Nicolas Heess, Thomas Degris, Daan Wierstra, and Martin Riedmiller. Deterministic policy gradient algorithms. In *International Conference on Machine Learning*, pages 387–395, 2014. 2

[39] Richard S. Sutton, David A. McAllester, Satinder P. Singh, and Yishay Mansour. Policy gradient methods for reinforcement learning with function approximation. In *Advances in Neural Information Processing Systems*, pages 1057–1063, 2000. 2

[40] Yonglong Tian, Andrew Luo, Xingyuan Sun, Kevin Ellis, William T. Freeman, Joshua B. Tenenbaum, and Jiajun Wu. Learning to infer and execute 3d shape programs. *arXiv*, 2019. Presented at ICLR 2019. 2

[41] Mikaela Angelina Uy, Yen yu Chang, Minhyuk Sung, Purvi Goel, Joseph Lambourne, Tolga Birdal, and Leonidas Guibas. Point2cyl: Reverse engineering 3d objects from point clouds to extrusion cylinders. In *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022. 2

[42] Aayush Vardhan, Rishabh Sahay, Abhishek Pandey, et al. Mcm: A mechanical components dataset for geometric deep learning. *arXiv preprint arXiv:2306.09053*, 2023. 2

[43] Ruiyu Wang, Yu Yuan, Shizhao Sun, and Jiang Bian. Text-to-cad generation through infusing visual feedback in large language models. *ICML*, 2025. 1, 2

[44] Siyu Wang, Cailian Chen, Xinyi Le, Qimin Xu, Lei Xu, Yanzhou Zhang, and Jie Yang. Cad-gpt: Synthesising cad construction sequence with spatial reasoning-enhanced multimodal llms. *arXiv preprint arXiv:2412.19663*, 2024. 1

[45] Siyu Wang, Cailian Chen, Xinyi Le, Qimin Xu, Lei Xu, Yanzhou Zhang, and Jie Yang. Cad-gpt: Synthesising cad construction sequence with spatial reasoning-enhanced multimodal llms. *AAAI*, 2025. Accepted at AAAI 2025 (Vol. 39, No. 8, pp. 7880–7888). 2

[46] Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8:229–256, 1992. 2

[47] Karl D. D. Willis, Yewen Pu, Jieliang Luo, Hang Chu, Tao Du, Joseph G. Lambourne, Armando Solar-Lezama, and Wojciech Matusik. Fusion 360 gallery: A dataset and environment for programmatic cad construction from human design sequences. *ACM Transactions on Graphics (TOG)*, 40(4):1–21, 2021. 2

[48] Q. Wu, K. Xu, and J. Wang. Constructing 3d csg models from 3d raw point clouds. *Computer Graphics Forum*, 37 (5), 2018. 2

[49] Rundi Wu, Chang Xiao, and Changxi Zheng. Deepcad: A deep generative network for computer-aided design models. *ICCV*, 2021. 1, 2, 4

[50] Sifan Wu, Amir Khasahmadi, Mor Katz, Pradeep Kumar Jayaraman, Yewen Pu, Karl Willis, and Bang Liu. Cad-llm: Large language model for cad generation. In *NeurIPS Workshop on Machine Learning for Creativity and Design*, 2023. Workshop, NeurIPS 2023, New Orleans, LA. 2

[51] Jingwei Xu, Zibo Zhao, Chenyu Wang, Wen Liu, Yi Ma, and Shenghua Gao. Cad-mllm: Unifying multimodality-conditioned cad generation with mllm. *arXiv preprint arXiv:2411.04954*, 2025. 2

[52] Xianghao Xu, Wenzhe Peng, Chin-Yi Cheng, Karl D. D. Willis, and Daniel Ritchie. Inferring CAD modeling sequences using zone graphs. *arXiv preprint arXiv:2104.03900*, 2021. Submitted 30 Mar 2021; revised 20 Apr 2021. 1, 2

[53] David Yarowsky. Unsupervised word sense disambiguation rivaling supervised methods. In *Proceedings of the 33rd Annual Meeting of the Association for Computational Linguistics*, pages 189–196, Cambridge, Massachusetts, USA, 1995. Association for Computational Linguistics. 2

[54] Licheng Zhang, Bach Le, Naveed Akhtar, Siew-Kei Lam, and Tuan Ngo. Large language models for computer-aided design: A survey. *arXiv preprint arXiv:2505.08137*, 2025. arXiv:2505.08137. 2

[55] Shengdi Zhou, Tianyi Tang, and Bin Zhou. Cadparser: A learning approach of sequence modeling for b-rep cad. In *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence (IJCAI)*, 2023. 1

[56] Haotian Zhu, Guyue Zhang, Zekun Hao, Zipeng Gao, Hengyang Zhao, Yifei Ren, Qingyang Wu, Xuan Luo, Jiahao Zhang, Masha Shugrina, and Xinchen Yan. Text2cad: A large-scale benchmark for language-driven cad modeling. *arXiv preprint arXiv:2409.17106*, 2024. 2

[57] Brian D. Ziebart, Andrew L. Maas, J. Andrew Bagnell, and Anind K. Dey. Maximum entropy inverse reinforcement learning. In *AAAI Conference on Artificial Intelligence*, pages 1433–1438, 2008. 2

[58] Barret Zoph, Golnaz Ghiasi, Tsung-Yi Lin, Yin Cui, Hanxiao Liu, Ekin D. Cubuk, and Quoc V. Le. Rethinking pre-training and self-training. *arXiv preprint arXiv:2006.06882*, 2020. 2